

DBMS Unit 3 - Interactive SQL and Performance Tuning

UNIT - III: INTERACTIVE SQL AND PERFORMANCE TUNING

MSBTE K-Scheme | Diploma CO | SY - Database Management System

3.1 SQL (STRUCTURED QUERY LANGUAGE)

SQL is a standard language used to communicate with a relational database. It is used to create, manipulate, control, and query data stored in a DBMS. SQL is **non-procedural** (declarative) — the user specifies **what** data is needed, not **how** to get it.

SQL Data Types

Category	Data Type	Description
Numeric	INT / INTEGER	Whole numbers
	SMALLINT	Small range whole numbers
	DECIMAL(p,s) / NUMERIC(p,s)	Fixed-point numbers with precision(p) and scale(s)
	FLOAT / REAL	Floating-point numbers
Character	CHAR(n)	Fixed-length character string of size n
	VARCHAR(n)	Variable-length character string, max size n
Date/Time	DATE	Stores date (YYYY-MM-DD)
	TIME	Stores time (HH:MM:SS)
	TIMESTAMP	Stores date and time together
Boolean	BOOLEAN	Stores TRUE/FALSE
Large Objects	BLOB	Binary Large Object (images, files)
	CLOB	Character Large Object (large text)

SQL commands are broadly classified into **five categories**:

1. Data Definition Language (DDL)

DDL commands are used to **define, create, or modify the structure** of database objects like tables, schemas, and indexes. Changes made by DDL commands are **auto-committed** (permanently saved).

Command	Purpose	Syntax Example
CREATE	Creates a new database object	CREATE TABLE Student (Roll_No INT, Name VARCHAR(20));
ALTER	Modifies the structure of an existing object	ALTER TABLE Student ADD Marks INT;
DROP	Permanently deletes an object and its structure	DROP TABLE Student;
TRUNCATE	Removes all records from a table but keeps its structure	TRUNCATE TABLE Student;
RENAME	Renames a database object	RENAME TABLE Student TO Students;

2. Data Manipulation Language (DML)

DML commands are used to **manipulate/manage data** stored within tables (insert, update, retrieve, delete). Changes can be **rolled back** since they are not auto-committed (unless auto-commit mode is on).

Command	Purpose	Syntax Example
SELECT	Retrieves data from table(s)	SELECT * FROM Student;
INSERT	Adds new record(s) into a table	INSERT INTO Student VALUES (1, 'Aditi', 85);
UPDATE	Modifies existing record(s)	UPDATE Student SET Marks = 90 WHERE Roll_No = 1;
DELETE	Removes record(s) from a table	DELETE FROM Student WHERE Roll_No = 1;

3. Data Control Language (DCL)

DCL commands are used to **control access/permissions** to the database (security and authorization).

Command	Purpose	Syntax Example
GRANT	Gives specific privileges to a user	GRANT SELECT, INSERT ON Student TO user1;
REVOKE	Withdraws previously granted privileges	REVOKE INSERT ON Student FROM user1;

4. Transaction Control Language (TCL)

TCL commands are used to **manage transactions** in the database, ensuring data consistency.

Command	Purpose	Syntax Example
	Permanently saves all changes made in	

COMMIT	the current transaction	COMMIT;
ROLLBACK	Undoes changes made in the current transaction (since last commit)	ROLLBACK;
SAVEPOINT	Sets a point within a transaction to which you can rollback later	SAVEPOINT sp1;
SET TRANSACTION	Sets properties for the transaction	SET TRANSACTION READ ONLY;

Quick Classification Summary:

Type	Full Form	Commands
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	GRANT, REVOKE
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT

3.2 CLAUSES & JOINS

Different Types of Clauses

1. WHERE Clause - Used to **filter records** based on a specified condition. - Applied to individual rows **before** grouping.

```
SELECT * FROM Student WHERE Marks > 75;
```

2. GROUP BY Clause - Used to **group rows** that have the same values in specified columns, typically used with aggregate functions (SUM, COUNT, AVG, etc.).

```
SELECT Branch, COUNT(*) FROM Student GROUP BY Branch;
```

3. ORDER BY Clause - Used to **sort** the result set in ascending (ASC, default) or descending (DESC) order.

```
SELECT * FROM Student ORDER BY Marks DESC;
```

4. HAVING Clause - Used to filter **groups** (used only with GROUP BY), unlike WHERE which filters individual rows. - Applied **after** grouping is done.

```
SELECT Branch, COUNT(*) FROM Student GROUP BY Branch HAVING COUNT(*) > 5;
```

WHERE vs HAVING:

Basis	WHERE	HAVING
Applied on	Individual rows	Groups (after GROUP BY)
Used with Aggregate	SELECT, UPDATE, DELETE	Only with GROUP BY

functions	Cannot be used	Can be used
-----------	----------------	-------------

Joins

A **JOIN** is used to combine rows from two or more tables based on a related column between them.

1. Inner Join - Returns only the rows that have **matching values** in both tables.

```
SELECT S.Name, D.Dept_Name
FROM Student S
INNER JOIN Department D ON S.Dept_ID = D.Dept_ID;
```

2. Left Outer Join (Left Join) - Returns **all rows from the left table**, along with matching rows from the right table. If no match, NULL is returned for right table columns.

```
SELECT S.Name, D.Dept_Name
FROM Student S
LEFT JOIN Department D ON S.Dept_ID = D.Dept_ID;
```

3. Right Outer Join (Right Join) - Returns **all rows from the right table**, along with matching rows from the left table. If no match, NULL is returned for left table columns.

```
SELECT S.Name, D.Dept_Name
FROM Student S
RIGHT JOIN Department D ON S.Dept_ID = D.Dept_ID;
```

4. Full Outer Join - Returns **all rows** from both tables, matched where possible; unmatched rows from either side get NULLs.

```
SELECT S.Name, D.Dept_Name
FROM Student S
FULL OUTER JOIN Department D ON S.Dept_ID = D.Dept_ID;
```

5. Self Join - A table is joined **with itself**, treated as two separate tables using aliases.

```
SELECT A.Name AS Employee, B.Name AS Manager
FROM Employee A, Employee B
WHERE A.Manager_ID = B.Emp_ID;
```

6. Cross Join - Returns the **Cartesian product** of two tables (every row of table A combined with every row of table B).

```
SELECT * FROM Student CROSS JOIN Department;
```

Summary Diagram (Venn representation):

INNER JOIN (AnB)	LEFT JOIN (All A)	RIGHT JOIN (All B)	FULL OUTER JOIN (AuB)
●●	●●	●●	●●
(overlap)	(left+overlap)	(overlap+right)	(everything)

Nested Queries (Subqueries)

A **nested query (subquery)** is a query written **inside another query**. The inner query executes first, and its result is used by the outer query.

Types of subqueries:

- **Single-row subquery** - returns only one row
- **Multi-row subquery** - returns multiple rows (used with IN, ANY, ALL)
- **Correlated subquery** - inner query depends on the outer query for its values

Example (single-row subquery):

```
SELECT Name FROM Student
WHERE Marks = (SELECT MAX(Marks) FROM Student);
```

Example (multi-row subquery using IN):

```
SELECT Name FROM Student
WHERE Dept_ID IN (SELECT Dept_ID FROM Department WHERE Dept_Name =
'CO');
```

3.3 OPERATORS

1. Relational (Comparison) Operators

Used to compare two values.

Operator	Meaning	Example
=	Equal to	Marks = 90
!= or <>	Not equal to	Marks != 90
>	Greater than	Marks > 50
<	Less than	Marks < 50
>=	Greater than or equal to	Marks >= 50
<=	Less than or equal to	Marks <= 50

2. Arithmetic Operators

Used to perform mathematical operations on numeric data.

Operator	Meaning	Example
+	Addition	Price + Tax
-	Subtraction	Price - Discount
*	Multiplication	Qty * Price
/	Division	Total / Count
%	Modulus (remainder)	Marks % 2

3. Logical Operators

Used to combine multiple conditions.

Operator	Meaning	Example
AND	True if both conditions are true	Marks > 50 AND Branch = 'CO'
OR	True if at least one condition is true	Marks > 90 OR Attendance > 90
NOT	Reverses the result of a condition	NOT (Branch = 'CO')
BETWEEN	Checks if value lies within a range	Marks BETWEEN 40 AND 80
LIKE	Pattern matching using wildcards (% , _)	Name LIKE 'A%'
IN	Checks if a value matches any in a list	Branch IN ('CO', 'IF')
IS NULL	Checks for NULL value	Marks IS NULL

4. Set Operators

Used to combine results of two or more SELECT queries. Both queries must have the **same number of columns** with **compatible data types**.

Operator	Description
UNION	Combines results of two queries, removes duplicate rows
UNION ALL	Combines results of two queries, keeps duplicate rows
INTERSECT	Returns only rows common to both queries
MINUS / EXCEPT	Returns rows present in the first query but not in the second

```
SELECT Name FROM Student_CO
UNION
SELECT Name FROM Student_IF;
```

3.4 FUNCTIONS

SQL functions perform calculations on data and return a single value.

1. Numeric Functions

Function	Description	Example
ABS(n)	Returns absolute value	ABS(-15) → 15
ROUND(n,d)	Rounds number to d decimal places	ROUND(12.567,2) → 12.57
TRUNC(n,d)	Truncates number to d decimal places	TRUNC(12.567,1) → 12.5
MOD(n,m)	Returns remainder of division	MOD(10,3) → 1
POWER(n,m)	Returns n raised to power m	POWER(2,3) → 8
SQRT(n)	Returns square root	SQRT(16) → 4
CEIL(n)	Rounds up to nearest integer	CEIL(4.2) → 5
FLOOR(n)	Rounds down to nearest integer	FLOOR(4.8) → 4

2. Date and Time Functions

Function	Description	Example
SYSDATE / NOW()	Returns current date and time	SELECT SYSDATE;
CURRENT_DATE	Returns current date	SELECT CURRENT_DATE;
ADD_MONTHS(d,n)	Adds n months to date d	ADD_MONTHS(hire_date,3)
MONTHS_BETWEEN(d1,d2)	Returns number of months between two dates	MONTHS_BETWEEN(date1,date2)

DATEDIFF(d1,d2)	Returns difference in days	DATEDIFF(date1,date2)
EXTRACT()	Extracts part of date (YEAR, MONTH, DAY)	EXTRACT(YEAR FROM SYSDATE)

3. String (Character) Functions

Function	Description	Example
UPPER(s)	Converts string to uppercase	UPPER('rahul') → RAHUL
LOWER(s)	Converts string to lowercase	LOWER('RAHUL') → rahul
LENGTH(s)	Returns length of string	LENGTH('Rahul') → 5
SUBSTR(s,start,len)	Extracts a substring	SUBSTR('Database',1,4) → Data
CONCAT(s1,s2)	Joins two strings	CONCAT('Data','base') → Database
TRIM(s)	Removes leading/trailing spaces	TRIM(' Rahul ') → 'Rahul'
REPLACE(s,old,new)	Replaces part of a string	REPLACE('ABCD','B','X') → AXCD
INSTR(s, sub)	Returns position of a substring	INSTR('Database','base') → 5

4. Aggregate Functions

Aggregate functions perform a calculation on a **set of rows** and return a **single summary value**. Commonly used with GROUP BY.

Function	Description	Example
COUNT()	Counts number of rows	COUNT(*)
SUM()	Returns total sum of a numeric column	SUM(Marks)
AVG()	Returns average value	AVG(Marks)
MAX()	Returns maximum value	MAX(Marks)
MIN()	Returns minimum value	MIN(Marks)

```
SELECT Branch, AVG(Marks) AS Avg_Marks
FROM Student
GROUP BY Branch;
```

3.5 VIEWS, SEQUENCES, INDEXES

VIEWS

A **view** is a **virtual table** based on the result of a SQL query. It does not store data physically; it is derived dynamically from one or more base tables each time it is accessed.

Purpose of Views: - Provides data security (hides sensitive columns/rows) - Simplifies complex queries - Provides a customized view of data for different users

1. Create View

```
CREATE VIEW CO_Students AS
```

```
SELECT Roll_No, Name, Marks
FROM Student
WHERE Branch = 'CO';
```

2. Update View (modify definition)

```
CREATE OR REPLACE VIEW CO_Students AS
SELECT Roll_No, Name, Marks, Branch
FROM Student
WHERE Branch = 'CO';
```

(Data inside a view can also be updated using UPDATE statement, if the view is updatable — i.e., based on a single table without aggregate functions/GROUP BY.)

3. Drop View

```
DROP VIEW CO_Students;
```

SEQUENCES

A **sequence** is a database object used to **automatically generate a series of unique numeric values**, typically used for primary key columns.

1. Create Sequence

```
CREATE SEQUENCE seq_roll
START WITH 1
INCREMENT BY 1
MAXVALUE 1000
NOCYCLE;
```

2. Alter Sequence

```
ALTER SEQUENCE seq_roll
INCREMENT BY 2
MAXVALUE 2000;
```

3. Drop Sequence

```
DROP SEQUENCE seq_roll;
```

4. Use of Sequence in Table

Sequences are accessed using **pseudo-columns**: - NEXTVAL - returns the next value in the sequence - CURRVAL - returns the current value of the sequence

```
INSERT INTO Student (Roll_No, Name)
VALUES (seq_roll.NEXTVAL, 'Rahul');
```

INDEXES

An **index** is a database object created on one or more columns of a table to **speed up data retrieval** operations, similar to an index in a book.

- Improves the performance of SELECT queries with WHERE/ORDER BY.
- Slightly slows down INSERT, UPDATE, DELETE since the index also needs to be updated.

Types of Index:

Type	Description
Unique Index	Ensures all values in the indexed column(s) are distinct

	(automatically created for Primary Key)
Composite Index	Created on two or more columns of a table
Clustered Index	Physically reorders the actual table data to match the index; only one per table
Non-Clustered Index	Creates a separate structure that points to the actual data rows; a table can have multiple
Bitmap Index	Uses bitmaps, efficient for columns with a small number of distinct values

1. Create Index

```
CREATE INDEX idx_name
ON Student (Name);
```

Create Unique Index:

```
CREATE UNIQUE INDEX idx_roll
ON Student (Roll_No);
```

2. Drop Index

```
DROP INDEX idx_name;
```

Quick Revision Points

- **DDL** = structure (CREATE, ALTER, DROP) | **DML** = data (SELECT, INSERT, UPDATE, DELETE)
- **DCL** = permissions (GRANT, REVOKE) | **TCL** = transactions (COMMIT, ROLLBACK, SAVEPOINT)
- **WHERE** filters rows before grouping; **HAVING** filters groups after GROUP BY
- **Joins:** Inner (matching only), Left/Right (all + matching), Full Outer (all), Self, Cross
- **Set Operators:** UNION (distinct), UNION ALL (with duplicates), INTERSECT, MINUS
- **Aggregate Functions:** COUNT, SUM, AVG, MAX, MIN — used with GROUP BY
- **View** = virtual table (query-based, no physical storage)
- **Sequence** = auto-generates unique numbers (NEXTVAL, CURRVAL)
- **Index** = speeds up retrieval; types include Unique, Composite, Clustered, Non-Clustered, Bitmap